Reviewer Pack — Secant Variety Dimension Lower Bounds for Disjointness

Entry 0fa75c46e35c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 01:19:58 UTC

Secant Variety Dimension Lower Bounds for Disjointness

Entry ID: `0fa75c46e35c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-01 01:19:58 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry **Field B** (complexity object): Communication Complexity

Statement:

> The dimension of the secant variety of the matrix M representing the disjointness function satisfies dim(σ₂(V(M))) ≥ n/2 for all n×n 0-1 matrices M with M[i][j] = 1 iff i ∩ j = ∅.

Rationale (proposer's reasoning):

> Secant varieties capture the geometry of linear combinations of points, which mirrors the information-theoretic constraints in distributed computation. The dimension lower bound translates to communication complexity via the relation between algebraic degeneracy and protocol efficiency.

Taxonomy category: `COMM_DISJ` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `331f0fd0b24351ec`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): `- secant variety disjointness function lower bound` `- secant variety matrix representation communication complexity` `- variety dimension communication complexity lower bounds`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.3s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    M = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]

    def rank(matrix):
        m, n = len(matrix), len(matrix[0])
        augmented_matrix = [row + matrix[i] for i, row in enumerate(matrix)]
        rows = list(range(m))
        cols = list(range(n))

        for col in cols:
            if all(row[col] == 0 for row in augmented_matrix):
                continue
            pivot_row = next(i for i, row in enumerate(augmented_matrix) if row[col] != 0)
            augmented_matrix[pivot_row], augmented_matrix[rows[0]] = augmented_matrix[rows[0]], au
            rows[0], rows[i] = rows[i], rows[0]

            for i in range(1, m):
                factor = -augmented_matrix[i][col] / augmented_matrix[0][col]
                augmented_matrix[i] = [factor * a + b for a, b in zip(augmented_matrix[0], augment

        return sum(1 for row in rows if any(row[col] != 0 for col in cols))

    def secant_variety_dimension(matrix):
        m, n = len(matrix), len(matrix[0])
        rank_2_flattening = []
        for i in range(m):
            for j in range(n):
                if matrix[i][j] == 1:
                    row = [matrix[x][y] * (x != i and y != j) for x in range(m) for y in range(n)]
                    rank_2_flattening.append(row)
        return rank(rank_2_flattening)

    dim_secant_variety = secant_variety_dimension(M)

    metric_name = "secant_variety_dimension"
```

```
    metric_value = dim_secant_variety
    instances_tested = 1
    conjecture_holds = dim_secant_variety >= n / 2
    counterexample = "" if conjecture_holds else f"dim(σ₂(V(M))) = {dim_secant_variety}, expected

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={fir
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0be076e5.py", line 75, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0be076e5.py", line 53, in run_trial
    dim_secant_variety = secant_variety_dimension(M)
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0be076e5.py", line 51, in secant_variet
    return rank(rank_2_flattening)
           ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0be076e5.py", line 35, in rank
    rows[0], rows[i] = rows[i], rows[0]
                       ^
```

```
UnboundLocalError: cannot access local variable 'i' where it is not associated with a value
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed due to unbound local variable error, preventing dimension calculation | next: Fix the rank computation code to handle row swaps properly and re-run experiments

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	86270
2	preregistration	ollama_remote	qwen3:8b	0	0	24101
3	novelty	ollama_remote	qwen3:8b	0	0	17687
4	novelty	ollama_remote	qwen3:8b	0	0	15583
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12788
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9438
7	judge	ollama_remote	qwen3:8b	0	0	18360

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 184225 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/0fa75c46e35c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0fa75c46e35c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0fa75c46e35c.tar.gz` (if generated)